

RAG para análise de incidentes de cibersegurança

1.1 Descrição do problema

Qual é o problema que você deseja resolver?

Uma equipe responsável pela segurança da informação precisa investigar diferentes tipos de incidentes, como falhas de segurança, acessos não autorizados, atividades suspeitas na rede, vazamento de dados, malware, entre outros.

Durante uma investigação, o analista precisa consultar vários documentos para entender o que aconteceu e quais procedimentos deverá realizar. Essas informações podem estar espalhadas em diferentes arquivos, como relatórios de incidentes, procedimentos de segurança, políticas internas e documentações técnicas.

O problema é que, quando existe uma grande quantidade de arquivos, encontrar uma informação específica pode levar bastante tempo, principalmente quando o analista precisa consultar vários documentos para encontrar uma resposta.

A proposta seria criar uma aplicação RAG que permita ao analista fazer perguntas em linguagem natural e receber respostas baseadas nos documentos da organização, facilitando a busca por informações sem precisar consultar vários arquivos manualmente.

Quem utilizaria a aplicação? Descreva o usuário concretamente: cargo, contexto de uso, nível técnico.

A aplicação seria utilizada por analistas de segurança da informação, principalmente profissionais de SOC (Security Operations Center). O usuário teria conhecimento técnico intermediário ou avançado e utilizaria o sistema durante a investigação de um incidente.

Security Operations Center (SOC) é um Centro de Operações de Segurança.

O SOC é uma equipe ou área responsável por monitorar, detectar, investigar e responder a ameaças e incidentes de segurança de uma organização.

Que tipo de informação o usuário gostaria de consultar?

- Relatórios de incidentes anteriores;
- Playbooks de resposta a incidentes;
- Políticas internas de segurança;
- Procedimentos de investigação;
- Relatórios de vulnerabilidades;
- Histórico de ocorrências;
- Informações sobre sistemas afetados;
- Ações realizadas em incidentes anteriores;
- Medidas tomadas para solucionar determinados problemas.

De onde vêm essas informações?

Os documentos utilizados pelo RAG seriam provenientes das próprias fontes da organização.

Por que utilizar um LLM sozinho não seria suficiente?

Um LLM sozinho pode não conhecer os **documentos e procedimentos internos da empresa**, além de poder gerar respostas que não estejam de acordo com eles.

Com RAG, o sistema busca informações nos documentos da organização e usa essas informações para gerar a resposta. Assim, o analista recebe uma resposta **mais específica e baseada nas fontes da empresa**, que também podem ser consultadas para verificar a informação.

Como o usuário vai utilizar o sistema?

A aplicação poderia ser disponibilizada por meio de uma interface web, semelhante a um chatbot interno da empresa.

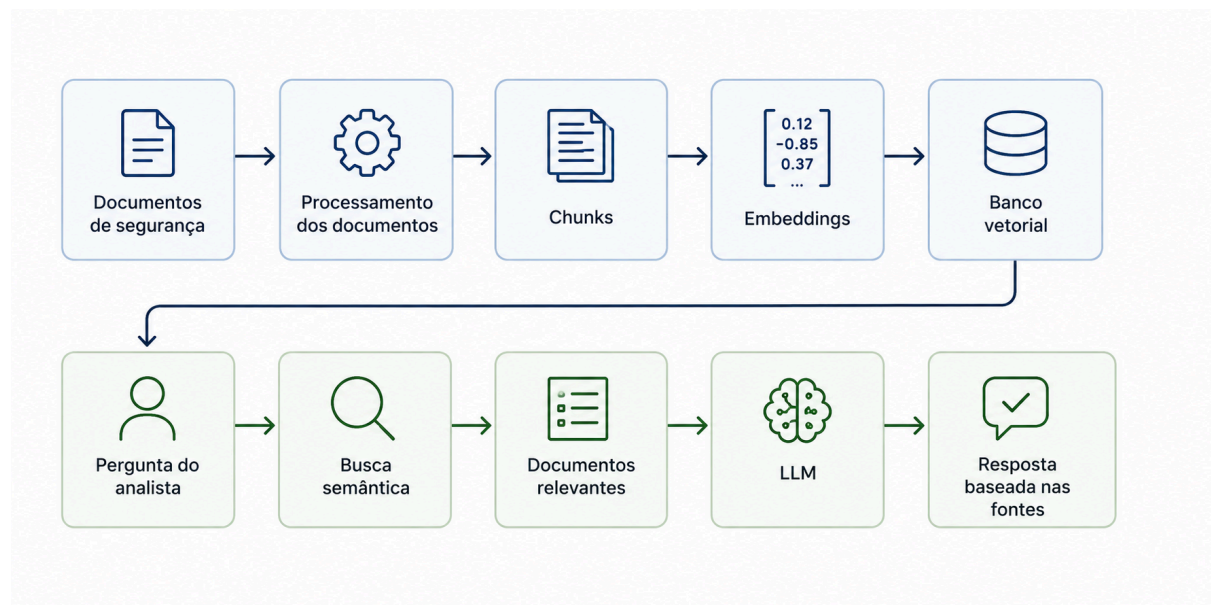
O analista acessaria a aplicação, escreveria uma pergunta e receberia uma resposta baseada nos documentos de segurança disponíveis.

Três perguntas reais que um usuário faria

Já tivemos um incidente parecido com esse antes?

Qual é o procedimento correto para esse tipo de ataque?

Qual procedimento devemos seguir quando uma conta é comprometida?



1.2 Por que RAG?

Por que RAG é adequado para esse problema?

O RAG é adequado porque o analista precisa consultar informações específicas e atualizadas da organização, como procedimentos de segurança, playbooks e incidentes anteriores. Em vez de depender apenas do conhecimento do LLM, o RAG busca essas informações nos documentos da empresa antes de gerar a resposta.

Que tipo de conhecimento precisa ser fornecido ao modelo?

- Playbooks de resposta a incidentes;
- Relatórios de incidentes anteriores;
- Políticas de segurança;
- Procedimentos internos;
- Relatórios de vulnerabilidades;
- Documentação dos sistemas e da infraestrutura.

Esse conhecimento muda com que frequência?

Pode mudar com bastante frequência, principalmente os relatórios de incidentes e informações sobre vulnerabilidades. Os playbooks e procedimentos também podem ser atualizados quando a empresa modifica suas políticas ou processos de segurança.

Existe necessidade de utilizar documentos privados ou específicos da organização?

Sim. O RAG precisa acessar documentos internos da empresa, como playbooks, relatórios de incidentes, políticas de segurança e procedimentos internos. Essas informações são específicas da organização e não fazem parte do conhecimento geral do LLM.

Que problemas poderiam ocorrer se o LLM respondesse apenas com seu conhecimento pré-treinado?

Se o LLM respondesse apenas com seu conhecimento pré-treinado, ele poderia dar respostas desatualizadas ou até incorretas, porque não teria acesso aos documentos e procedimentos específicos da empresa.

Exemplo concreto:

“O que devo fazer quando identifico uma conta comprometida?”

O LLM poderia responder de forma geral que o analista deve bloquear a conta e redefinir a senha.

Porém, o procedimento interno da empresa pode determinar que, antes de bloquear a conta, o analista deve preservar determinadas evidências e registrar o incidente em um sistema específico.

1.3 Limitações - quando RAG não é a resposta

Busca por palavras-chave:

É mais indicada quando o analista já sabe exatamente o que deseja encontrar. Por exemplo, ao pesquisar uma vulnerabilidade pelo seu código, como **CVE-2026-XXXX**, a busca consegue localizar diretamente as informações relacionadas àquele termo.

Banco de dados e SQL:

É uma opção melhor para trabalhar com dados que estão organizados e estruturados. Ele facilita tarefas como contar, filtrar e comparar informações. Por exemplo, descobrir quantos incidentes de segurança aconteceram durante um determinado mês.

Regras predefinidas:

São mais eficientes quando já existe uma ação determinada para cada situação. Por exemplo, se o sistema identificar uma condição específica, ele pode realizar automaticamente uma ação que já foi definida pela equipe de segurança.

Uso conjunto com RAG:

As tecnologias também podem trabalhar juntas. O banco de dados pode armazenar e consultar informações estruturadas, enquanto o RAG pode buscar informações nos documentos da empresa. Depois, o LLM pode reunir essas informações e apresentar uma resposta de forma mais clara para o analista.

Existe alguma pergunta que o RAG responderia mal e um banco relacional responderia bem?

“Quantos incidentes de acesso não autorizado aconteceram nos últimos 6 meses?”

Essa pergunta seria melhor respondida por um **banco de dados relacional**, porque envolve contar registros e aplicar um filtro por período e tipo de incidente.

O RAG poderia encontrar documentos que mencionem esses incidentes, mas teria mais dificuldade para garantir uma contagem exata.

E se a pergunta exigisse contar, somar ou ordenar informações espalhadas por muitos documentos?

O RAG poderia recuperar vários documentos relacionados, mas não é a ferramenta ideal para realizar cálculos exatos sobre grandes quantidades de dados. Poderiam ocorrer erros de contagem, informações duplicadas ou documentos não recuperados pela busca.

Nesse caso, seria melhor utilizar um banco de dados estruturado ou uma ferramenta de processamento de dados para realizar a contagem, soma ou ordenação. O RAG poderia ser utilizado junto com essas ferramentas para consultar os documentos e explicar os resultados.

Parte 2 - Organização dos documentos

Quais tipos de arquivo existirão? (PDF, DOCX, HTML, Markdown, páginas web, planilhas, imagens, áudios, vídeos, outros)

PDF: relatórios de incidentes, políticas e documentos técnicos;

DOCX: procedimentos internos e playbooks;

Markdown: documentações técnicas;

HTML: documentações de sistemas e ferramentas;

Planilhas: registros e históricos de incidentes.

Qual o volume aproximado? (dezenas, centenas, milhares de documentos?)

Inicialmente, a aplicação poderia trabalhar com centenas de documentos, por exemplo, entre 300 e 1.000 documentos.

Esse número poderia aumentar conforme a empresa acumulasse novos relatórios e procedimentos.

Qual o tamanho típico de cada documento? (Páginas, kbs)

A maioria dos documentos poderia ter entre 5 e 30 páginas, dependendo do tipo.

Por exemplo:

Playbooks: 5–15 páginas;

Políticas: 10–30 páginas;

Relatórios de incidentes: 5–20 páginas;

Documentações técnicas: 10–30 páginas.

Com que frequência novos documentos entram? Documentos antigos são atualizados ou substituídos?

Novos documentos poderiam entrar semanalmente ou diariamente, principalmente novos relatórios de incidentes.

Já documentos como políticas e playbooks seriam atualizados com menos frequência, por exemplo, mensalmente ou quando houver alguma mudança no procedimento.

Documentos antigos não seriam simplesmente apagados. Eles seriam mantidos como versões anteriores, mas identificados como obsoletos, para evitar que o RAG utilize uma versão antiga como resposta atual.

Organização das pastas

```
documentos/  
├── incidentes/  
├── playbooks/  
├── politicas/  
├── vulnerabilidades/  
├── procedimentos/  
└── documentacao_tecnica/
```

Essa divisão foi definida considerando as principais necessidades do analista durante a investigação e resposta a um incidente.

Cada tipo de documento possui uma finalidade. Playbooks e procedimentos orientam sobre como agir, enquanto relatórios de incidentes anteriores ajudam a entender casos já ocorridos e como foram resolvidos. Políticas e normas mostram as regras que devem ser seguidas.

Essa organização também facilita a aplicação de filtros, como:

- Tipo de documento
- Tipo de incidente
- Sistema afetado
- Data
- Versão
- Status do documento

Assim, o analista consegue encontrar mais rapidamente as informações relevantes para cada incidente.

Existe documento que não deve entrar na base? (informação sigilosa, dado pessoal, versão obsoleta) Como você impediria a entrada?

Sim. Nem todos os documentos da empresa devem entrar na base do RAG. Arquivos com senhas, chaves de API, tokens, dados pessoais ou informações sigilosas, além de documentos obsoletos ou corrompidos, devem ser excluídos. Para isso, pode existir uma etapa de validação antes da entrada na base, na qual os arquivos são classificados, dados sensíveis são removidos e documentos antigos são identificados antes do processamento.

Como você lidaria com versões do mesmo documento? Se a política de férias mudou em 2026, o sistema pode recuperar a versão de 2024 e responder errado.

Como lidar com versões do mesmo documento?

Cada documento deveria possuir controle de versão e data de atualização nos metadados.

Por exemplo:

```
documento: playbook_phishing
versao: 3.0
data: 2026-08-10
status: atual
```

Enquanto a versão antiga ficaria:

```
documento: playbook_phishing
versao: 2.0
data: 2024-05-15
status: obsoleto
```

Na hora da busca, o sistema deveria priorizar documentos com status “atual” e a versão mais recente.

Assim, se o procedimento de resposta a phishing mudou em 2026, o sistema não deveria recuperar a versão de 2024 como principal fonte da resposta.

Parte 3 - Pipeline de ingestão

```
Documentos
↓
Extração
↓
Limpeza / normalização (principalmente se o documento chegar
poluído demais)
↓
Metadados (Lembram dos output estruturados do llm?)
↓
Chunking / Splitting
↓
Embeddings
↓
Banco vetorial (até agora trabalhamos com uma lista de
vetores/embeddings que estamos fazendo a pesquisa na mão).
```

3.1 Extração

Como o texto seria extraído?

A extração dependerá do tipo de arquivo. Para PDFs, DOCX, Markdown e HTML, seriam utilizadas ferramentas específicas para transformar o conteúdo em texto estruturado, preservando, sempre que possível, informações como títulos, parágrafos, páginas e tabelas.

O processo adota uma abordagem orientada pelo tipo MIME e extensão de cada arquivo:

- Markdown (.md) e HTML: Extração direta via parsers estruturais (como [BeautifulSoup](#) ou leitores nativos de Markdown), preservando a hierarquia de tags (`#`, `##`, `<article>`, `<table>`).
- DOCX: Extração com bibliotecas como [python-docx](#) ou [unstructured](#), mapeando estilos de parágrafo (Título 1, Título 2, listas com marcadores) para Markdown equivalente.
- Planilhas (.csv, .xlsx): Conversão linha a linha ou bloco a bloco em representações estruturadas (formato Markdown Table ou formato chave-valor [Campo: Valor](#)).
- PDFs e Formatos Compostos: Roteamento condicional dependendo da presença de texto digital ou imagens.

Como você trataria PDFs com texto selecionável?

Em PDFs que possuem uma camada de texto, o conteúdo poderia ser extraído diretamente por ferramentas de parsing de documentos, mantendo informações como o texto, número da página e estrutura do documento.

```
Identificação do tipo de arquivo (MIME)
↓
Leitura do PDF
↓
Extração do texto
```

```
↓
Preservação de estrutura
↓
Limpeza/normalização
↓
Metadados
↓
Chunking
↓
Embeddings
```

E PDFs digitalizados (imagem escaneada, sem camada de texto)?

Para PDFs digitalizados, o processo é diferente porque o PDF contém imagens das páginas, e não texto que possa ser selecionado. Nesse caso, seria necessário utilizar OCR (Optical Character Recognition) para reconhecer os caracteres presentes nas imagens.

O processo seria:

```
PDF digitalizado
↓
Identificação das páginas como imagem
↓
OCR
↓
Texto extraído
↓
Limpeza e normalização
↓
Metadados
↓
Chunking
↓
Embeddings
↓
Banco vetorial
```

Como trataria tabelas? (é importante manter?)

As tabelas seriam extraídas preservando, sempre que possível, a relação entre linhas e colunas. Depois, poderiam ser convertidas para um formato estruturado, como Markdown, facilitando o processamento pelo RAG. Em PDFs, ferramentas de extração poderiam identificar a estrutura da tabela. Caso o PDF fosse digitalizado, seria necessário utilizar OCR com suporte à identificação de tabelas.

É importante manter as tabelas quando elas possuem informações relevantes para a investigação, como datas, sistemas afetados, vulnerabilidades, níveis de severidade, incidentes e ações realizadas. Isso evita que informações importantes sejam perdidas

durante a extração. Já tabelas apenas decorativas ou sem informações relevantes poderiam ser descartadas.

Um problema possível é a perda da estrutura da tabela durante a extração, fazendo com que os valores das colunas fiquem desorganizados ou sem relação com seus respectivos campos.

```
Tabela no documento
  ↓
Extração da estrutura
  ↓
Preservação de linhas e colunas
  ↓
Conversão para formato estruturado
  ↓
Limpeza
  ↓
Chunking
  ↓
Embeddings
```

Como trataria as imagens? Posso descartar? Quais informações elas têm?

As imagens seriam analisadas antes de serem descartadas. Primeiro, seria verificado se possuem informações relevantes, como diagramas de arquitetura, fluxogramas, topologias de rede, gráficos ou capturas de tela. Quando necessário, seria utilizado OCR ou análise visual para extrair essas informações. Imagens apenas decorativas, como logotipos e ícones, poderiam ser descartadas. Assim, seriam mantidas apenas as imagens que contribuíssem para a investigação e para as respostas do RAG.

```
Imagem
  ↓
Análise da imagem
  ↓
Identificação de conteúdo relevante
  ↓
OCR / Análise visual
  ↓
Extração das informações
  ↓
Limpeza e organização
  ↓
Metadados
  ↓
Armazenamento
```

Como trataria documentos multimodais?(multimodais = texto + imagem, audio + video, texto + video e etc)

Os documentos multimodais seriam tratados de acordo com cada tipo de conteúdo presente. Textos seriam extraídos normalmente, imagens poderiam passar por OCR ou análise visual, áudios seriam transcritos e vídeos poderiam ter o áudio transcrito e, quando

necessário, alguns frames analisados. Depois, as informações seriam organizadas e associadas ao documento por meio de metadados, mantendo a relação entre os diferentes tipos de conteúdo. Assim, o RAG poderia utilizar informações provenientes de texto, imagens, áudios e vídeos durante a busca.

```
Documento multimodal
↓
Identificação do conteúdo
↓
Extração
↓
Texto → extração direta
Imagem → OCR/análise visual
Áudio → transcrição
Vídeo → transcrição/análise
↓
Organização
↓
Metadados
↓
Chunking
↓
Embeddings
↓
Banco vetorial
```

Explique quais problemas podem surgir durante a extração. Se você já enfrentou algum deles nas atividades anteriores, cite o caso concreto.

Alguns problemas possíveis são:

- PDF com texto desorganizado ou fora da ordem;
- OCR com erros de reconhecimento;
- tabelas que perdem sua estrutura durante a extração;
- imagens contendo informações importantes que não são capturadas pelo parser;
- documentos com diferentes formatos e codificações;
- arquivos corrompidos ou protegidos;
- cabeçalhos e rodapés sendo repetidos várias vezes;
- conteúdo duplicado;
- documentos multimodais que exigem diferentes técnicas de processamento.

Um problema semelhante já apareceu nas atividades anteriores com a utilização do Docling para conversão de PDFs para Markdown. Durante a atividade, ocorreu um erro relacionado ao ambiente de execução, `InvalidCxxCompiler: Compiler: cl is not found`, indicando que o compilador necessário não estava disponível. Esse caso mostra que, além dos problemas relacionados ao conteúdo dos documentos, também podem existir problemas de dependências e configuração das ferramentas utilizadas na etapa de extração.

3.2 Limpeza e normalização

O que precisa ser removido? (cabeçalhos e rodapés repetidos, numeração de página, marcas d'água, sumário, referências)

Durante a limpeza, podem ser removidos elementos que não contribuem diretamente para a compreensão do conteúdo, como cabeçalhos e rodapés repetidos, numeração de páginas, marcas d'água e conteúdos duplicados. O sumário também pode ser removido quando o conteúdo completo dos títulos já estiver presente no documento. As referências devem ser avaliadas antes de serem removidas, pois podem conter informações importantes para determinadas consultas.

O que precisa ser padronizado? (acentuação, quebras de linha, espaçamento, codificação)

O texto pode ser padronizado para corrigir quebras de linha desnecessárias, espaçamentos excessivos, caracteres incorretos e problemas de codificação. Também é importante preservar a acentuação e a estrutura de títulos, listas e parágrafos.

Que informação você corre o risco de perder ao limpar demais?

Ao realizar uma limpeza muito agressiva, existe o risco de eliminar informações que parecem desnecessárias, mas que possuem significado para a investigação. Por exemplo, podemos perder relações entre informações de uma tabela, detalhes de um procedimento, nomes de sistemas, versões de documentos ou informações presentes em notas e rodapés. Por isso, a limpeza deve preservar o contexto original sempre que ele puder ser relevante para a busca.

3.3 Frequência de ingestão

O pipeline roda uma vez, sob demanda, ou de forma agendada? Com que frequência chegam novos documentos?

O pipeline seria executado de forma agendada, com verificações diárias, pois em um ambiente de cibersegurança podem surgir novos relatórios de incidentes, alertas, vulnerabilidades e registros de investigações com frequência. Também poderia ser executado sob demanda quando um incidente importante fosse encerrado e seu relatório precisasse ser disponibilizado rapidamente para futuras consultas.

Quando um documento é atualizado, você reprocessa só ele ou a base inteira? Como sabe qual reprocessar?

Quando um relatório de incidente, playbook ou procedimento de resposta fosse atualizado, apenas esse documento seria reproprocessado. O sistema poderia identificar a alteração por meio de ID do documento, versão, data de atualização e status. A versão anterior seria marcada como obsoleta, enquanto a nova versão teria seus chunks e embeddings atualizados no banco vetorial.

Parte 4 - Metadados

4.1 Metadados do documento

```
{
  "document_id": "PB-RANSOM-01",
  "title": "Playbook de Resposta a Incidentes de Ransomware",
  "document_type": "playbook",
  "classification": "confidencial",
  "version": "3.2",
  "status": "atual",
  "created_at": "2024-05-10T14:30:00Z",
  "updated_at": "2026-02-15T09:00:00Z",
  "source_path": "documentos/playbooks/PB-01_ransomware.md",
  "target_systems": ["Active Directory", "Storage Backup",
"Windows Server"],
  "threat_type": "ransomware"
}
```

Por que cada metadado é importante?

document_id	Identifica o documento de forma única e permite relacionar os chunks ao documento original.
title	Facilita a identificação do documento e pode ser apresentado ao analista como fonte da resposta.
document_type	Identifica se é um relatório de incidentes, playbook, política, procedimento, relatório de vulnerabilidade etc. É importante para filtrar a busca.
classification	Nível de confidencialidade(publico, interno, confidencial, restrito-soc). Garante controle de acesso (RBAC).
version	Controla diferentes versões de playbooks, procedimentos e políticas.
status	Indica se o documento está atual, obsoleto ou arquivado, evitando que uma versão antiga seja utilizada como principal fonte.
created_at	Indica quando o documento foi criado e ajuda a contextualizar incidentes anteriores.
updated_at	Permite identificar quando a informação foi atualizada e ajuda a priorizar documentos mais recentes.
source_path	Caminho de origem do arquivo bruto para auditoria e download direto pelo analista.
target_systems	Sistemas e tecnologias abordados. Permite filtrar o contexto por infraestrutura afetada.
threat_type	: Categoria do vetor de ataque (ex.: ransomware, phishing, ddos, brute_force), acelerando a

	recuperação temática.
--	-----------------------

4.2 Metadados do chunk

```
{
  "document_id": "123",
  "chunk_id": "123-05",
  "page": 5,
  "section": "Resposta ao incidente",
  "document_type": "playbook",
  "incident_type": "phishing",
  "affected_system": "e-mail",
  "version": "3.0",
  "status": "atual",
  "text": "..."
```

Por que cada metadado é importante?

document_id	Relaciona o chunk ao documento original.
chunk_id	Identifica individualmente cada trecho e facilita sua recuperação e gerenciamento.
page	Permite localizar a informação diretamente no documento original.
section	Indica a seção de onde o trecho foi retirado e ajuda a preservar o contexto.
document_type	Permite diferenciar, por exemplo, um procedimento de um relatório de incidente.
incident_type	Permite recuperar somente chunks relacionados ao tipo de incidente pesquisado.
affected_system	Permite filtrar informações relacionadas ao sistema envolvido no incidente.
version	Ajuda a garantir que o chunk pertence à versão correta do documento.
status	Permite evitar a recuperação de chunks pertencentes a documentos obsoletos.
text	Indica se o documento está atual, obsoleto ou arquivado, evitando que uma versão antiga seja utilizada como principal fonte.

Quais metadados você usaria para filtrar a busca? Dê um exemplo de pergunta em que o filtro é indispensável.

Eu usaria principalmente os metadados `document_type`, `incident_type`, `affected_system`, `version` e `status`. Eles permitem restringir a busca para documentos relacionados ao incidente e evitar versões antigas ou informações de outros contextos.

“Qual é o procedimento atual para responder a um incidente de phishing no sistema de e-mail?”

```
document_type = playbook
incident_type = phishing
affected_system = e-mail
status = atual
```

Quais metadados você usaria para citar a fonte ao usuário? O que exatamente apareceria na tela junto da resposta?

Para citar a fonte ao usuário, seriam utilizados principalmente os metadados `title`, `version`, `status`, `updated_at`, `source_path` e `document_id`.

Na tela, junto à resposta, poderia aparecer: Fonte: Playbook de Resposta a Incidentes de Ransomware — versão 3.2 (atual), última atualização em 15/02/2026, documento PB-RANSOM-01, origem: `documentos/playbooks/PB-01_ransomware.md`.

Que metadado seria caríssimo de acrescentar depois que a base já estivesse indexada? Por quê?

O `incident_type` seria um dos metadados mais trabalhosos de adicionar depois que a base já estivesse indexada. Isso porque seria necessário voltar aos documentos e analisar os chunks para identificar qual tipo de incidente cada um representa, como `ransomware`, `phishing` ou `malware`. Em uma base grande, esse processo poderia exigir bastante reprocessamento. Por isso, é melhor definir esse metadado já na etapa de ingestão.

Como você vai extrair esses metadados?

Os metadados seriam extraídos durante a etapa de ingestão, combinando informações disponíveis no próprio arquivo com informações identificadas no conteúdo. Dados como `document_id`, `title`, `version`, `created_at`, `updated_at` e `source_path` poderiam ser obtidos diretamente do arquivo ou definidos pelo sistema. Já informações como `threat_type`, `target_systems` e `document_type` poderiam ser identificadas a partir do conteúdo do documento, utilizando regras, classificação automática ou um LLM. Depois, esses dados seriam validados e associados ao documento e aos seus chunks antes da geração dos embeddings.

Parte 5 - Chunking / Splitting

Qual estratégia de splitting você utilizaria?

Splitter recursivo, tentando respeitar a estrutura do documento.

Qual tamanho aproximado dos chunks?

Aproximadamente 500 a 800 tokens por chunk.

Utilizaria overlap? Quanto?

Cerca de 10% a 15%, aproximadamente 50–100 tokens.

A divisão seria por caracteres, palavras, sentenças, parágrafos ou seções?

Priorizaria **seções** → **parágrafos** → **sentenças**, e só usaria caracteres como último recurso.

Utilizaria um splitter recursivo?

Sim. Ele tenta primeiro dividir em partes maiores, como seções e parágrafos, e só fragmenta mais quando o conteúdo ainda está grande.

Utilizaria uma estratégia específica para cada tipo de documento? Um contrato e uma transcrição de call center pedem o mesmo tratamento?

Sim. Os documentos de segurança podem ter estruturas muito diferentes:

Playbook	Dividir por seções e procedimentos.
Relatório de incidente	Dividir por etapas do incidente, como detecção, análise, contenção e recuperação.
Política de segurança	Dividir por seções e subseções.
Relatório técnico	Dividir por tópicos e parágrafos.
Transcrição de call	Dividir por blocos de conversa/assuntos, mantendo contexto entre falas
Tabelas	Preservar a tabela inteira ou dividi-la mantendo cabeçalho e contexto
Imagens/diagramas	Não dividir como texto; associar a imagem à seção correspondente

Um contrato e uma transcrição de call center não deveriam receber exatamente o mesmo tratamento, e o mesmo vale para documentos de cibersegurança.

O que pode acontecer se os chunks forem muito pequenos?

Perda de contexto;
Respostas incompletas;

Dificuldade para o RAG entender a relação entre as informações;
Maior quantidade de chunks para armazenar e buscar;
Possibilidade de o sistema recuperar um trecho correto, mas insuficiente para responder à pergunta.

O que pode acontecer se os chunks forem muito grandes?

O chunk pode conter informações demais e assuntos diferentes. Isso pode prejudicar a busca semântica, porque o resultado recuperado pode ter a informação procurada misturada com vários conteúdos irrelevantes.

Como você trataria uma tabela na hora de dividir? Uma tabela cortada ao meio ainda significa alguma coisa? e uma imagem?

Tabelas: evitaria cortar uma tabela no meio, porque uma parte isolada pode perder o significado das colunas, linhas e valores. O ideal seria manter a tabela inteira em um chunk ou, se ela for muito grande, dividi-la por linhas, repetindo o cabeçalho em cada parte.

Imagens: não faria um corte aleatório. Primeiro verificaria se a imagem contém informação importante, como diagramas, fluxos, gráficos ou evidências de um incidente. Se tiver, manteria a imagem associada ao texto que a explica ou extrairia uma descrição dela para o RAG.

Como saber se o chunking ficou bom?

Eu faria uma avaliação usando perguntas reais de um analista de segurança.

Por exemplo:

- “Como devemos conter um incidente de ransomware?”
- “Qual foi o sistema afetado nesse incidente?”
- “Quais indicadores de comprometimento foram encontrados?”
- “Qual procedimento deve ser realizado após detectar uma conta comprometida?”

Para cada pergunta, verificaria:

1. Se o chunk correto foi recuperado.
2. Se ele contém todo o contexto necessário para responder.
3. Se os resultados mais relevantes aparecem entre os primeiros resultados.
4. Se o LLM consegue responder corretamente usando apenas os chunks recuperados.

Parte 6 - Embeddings

Modelo escolhido	BAAI/bge-m3
Dimensão do embedding	1024
Suporta português?	Sim, por ser multilíngue e suportar

	mais de 100 idiomas.
É multilíngue?	Sim
Tamanho máximo de entrada	8.192 tokens
É open source?	Sim, licença MIT
Pode ser executado localmente?	Sim
Possui API?	Pode ser utilizado por provedores de inferência, além de execução local.
Custo aproximado	Gratuito para execução local; o custo passa a depender da infraestrutura utilizada.
Custo aproximado	Gratuito para execução local; o custo passa a depender da infraestrutura utilizada.

Fonte: https://huggingface.co/BAAI/bge-m3?utm_source

Por que esse modelo é adequado ao seu cenário?

O BGE-M3 é uma boa opção para o RAG de incidentes de cibersegurança porque consegue lidar com conteúdos em português e inglês, além de termos técnicos, códigos de vulnerabilidades e indicadores de comprometimento. Como suporta até 8.192 tokens, também permite trabalhar com textos maiores sem precisar dividi-los em partes muito pequenas. Outro ponto positivo é a possibilidade de execução local, o que pode ser importante para manter protegidas informações sensíveis presentes nos documentos de incidentes.

Considerou algum modelo alternativo e descartou? Qual, e por quê?

Eu consideraria o Cohere Embed v4, que possui API e permite embeddings de diferentes dimensões, incluindo 256, 512, 1024 e 1536. Porém, para esse cenário eu escolheria o BGE-M3 porque a possibilidade de execução local é mais interessante quando existe preocupação com sigilo dos documentos. A API da Cohere também possui cobrança por tokens em uso de produção.

Se o cenário envolve documentos sigilosos, isso muda sua escolha entre modelo local e API? Como?

Sim. Se os relatórios de incidentes tiverem informações confidenciais, eu daria preferência a um modelo local, pois os documentos não precisam ser enviados para um serviço externo de API. O BGE-M3 permite essa abordagem.

O tamanho máximo de entrada do modelo tem relação com a sua decisão de chunking da Parte 5? Explique.

Sim. O limite de 8.192 tokens mostra quanto texto o modelo consegue receber em uma única entrada. Como os chunks terão entre 500 e 800 tokens, haverá bastante espaço dentro desse limite. Isso permite manter as informações importantes juntas, sem precisar criar trechos excessivamente grandes.

Um diagrama do sistema completo, do documento original até a resposta ao usuário. Pode ser desenho, ferramenta de diagramação ou ASCII - o que importa é estar legível e completo.

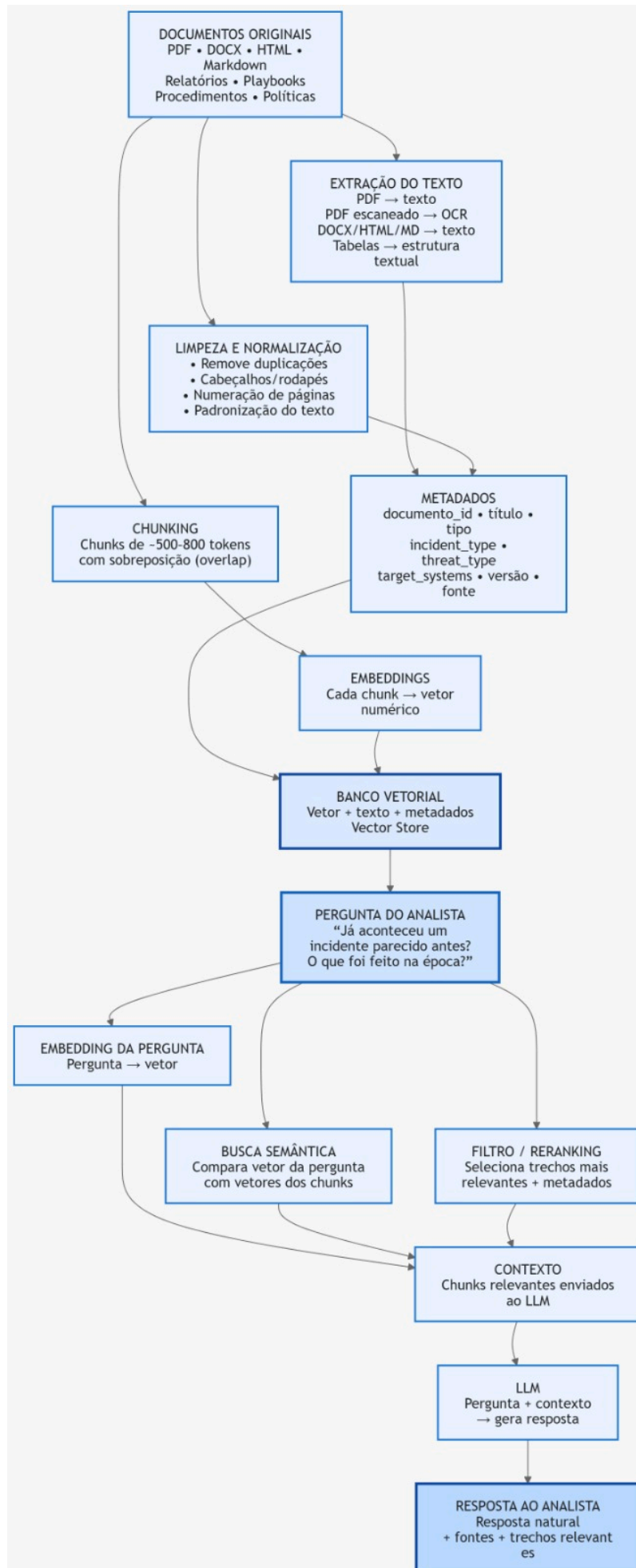


Tabela de decisões

Etapa	Decisão	Justificativa em uma linha
Extração	Extrair texto de PDFs, DOCX, HTML e Markdown; usar OCR em PDFs escaneados.	Permite transformar diferentes formatos em texto pesquisável, sem perder informações importantes.
Limpeza	Remover cabeçalhos/rodapés repetidos, numeração de páginas, marcas d'água e duplicações.	Melhora a qualidade do conteúdo que será utilizado pelo RAG, facilitando a identificação das informações realmente relevantes.
Chunking	Usar Recursive Character Text Splitter, com chunks de aproximadamente 500-800 tokens e pequeno overlap.	Mantém o contexto dos incidentes sem criar trechos grandes demais ou pouco precisos.
Metadados	documento_id, título, tipo, incident_type, target_systems, version e chunk_index	Facilita filtros, organização e recuperação mais precisa dos documentos.
Embeddings	Usar BAAI/bge-m3, com embeddings de 1024 dimensões.	É multilíngue, suporta português e inglês e funciona bem para busca semântica em documentos de segurança.

Riscos e limitações da sua própria proposta. O que você sabe que essa arquitetura não resolve bem?

Qualidade dos documentos: se os documentos estiverem incompletos, incorretos ou desatualizados, o RAG poderá recuperar essas informações e gerar uma resposta inadequada.

Falha na recuperação: o sistema pode não localizar a informação correta devido a termos diferentes ou à divisão inadequada dos chunks.

Informações desatualizadas: uma versão antiga do documento pode ser recuperada se os metadados ou filtros não forem aplicados corretamente.

Tabelas, imagens e documentos complexos: a extração pode não preservar corretamente a organização das informações, dificultando a compreensão do conteúdo.

Contagens e cálculos: o RAG pode apresentar dificuldades em cálculos e contagens exatas, sendo mais adequado usar um banco de dados ou ferramenta específica.

Não substitui o analista: o sistema serve como apoio à consulta, mas as decisões durante o incidente devem ser tomadas pelo profissional responsável.

Risco de respostas incorretas: mesmo com informações relevantes, o LLM pode interpretar ou combinar os dados de forma errada.

Segurança e controle de acesso: como os documentos podem conter dados confidenciais, é necessário garantir que cada usuário acesse apenas as informações permitidas.

Custo e manutenção: o crescimento da base aumenta os custos de armazenamento e processamento, além da necessidade de atualizar documentos e embeddings.